DevOps: tư duy và vòng đời dịch vụ

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

DevOps giải quyết vấn đề gì?

Một ứng dụng không kết thúc khi lập trình viên viết xong mã. Mã phải được kiểm tra, đóng gói, đưa lên môi trường chạy, cấu hình, giám sát, sửa khi có lỗi và cập nhật nhiều lần. Nếu mỗi bước chỉ nằm trong đầu một người, việc phát hành chậm và dễ sai. DevOps là tập hợp cách làm giúp nhóm phát triển và vận hành phối hợp để đưa thay đổi tới người dùng một cách thường xuyên, an toàn và có thể quan sát được. Từ này không chỉ tên một phần mềm hay một chứng chỉ.

Hãy tưởng tượng một dịch vụ ghi chú. Người dùng bấm Lưu nhưng dữ liệu không xuất hiện. Lỗi có thể nằm ở giao diện, API, database, mạng, cấu hình, quyền truy cập hoặc bản phát hành vừa được đưa lên. Người làm DevOps cần hiểu đường đi của thay đổi và của một request, biết xem tín hiệu, thu hẹp phạm vi nghi vấn và khôi phục dịch vụ. Họ thường xây công cụ tự động để nhóm tránh lặp lỗi cũ.

Một vòng đời đơn giản là: lập kế hoạch -> viết mã -> kiểm tra -> đóng gói -> phát hành -> vận hành -> quan sát -> học từ phản hồi. Các mũi tên quan trọng như từng ô: nếu một bản sửa đã được kiểm tra nhưng người vận hành không biết phiên bản nào đang chạy, vòng đời vẫn có lỗ hổng. Nếu có dashboard đẹp nhưng không thể rollback, sự cố vẫn kéo dài.

Ví dụ có giải thích

Giả sử phiên bản 1 trả về danh sách ghi chú. Phiên bản 2 thêm chức năng tìm kiếm. Trước khi phát hành, nhóm chạy test; nếu test qua, pipeline tạo một artifact có định danh; hệ thống triển khai artifact ấy; một smoke test xác nhận API còn trả kết quả; người vận hành nhìn lỗi và độ trễ; nếu nhiều request thất bại, hệ thống có thể quay về phiên bản 1. Mỗi bước có bằng chứng. Khả năng trả lời "đang chạy phiên bản nào?", "lỗi xuất hiện từ khi nào?" và "làm sao phục hồi?" là dấu hiệu của quy trình tốt.

Tự kiểm tra

- Vì sao một ứng dụng chạy được trên máy của lập trình viên vẫn có thể lỗi khi triển khai?
- Tự động hóa giúp ích gì nếu thiếu kiểm tra và quan sát?
- Trong ví dụ trên, artifact khác mã nguồn ở điểm nào?

Gợi ý: artifact là kết quả build có thể triển khai, ví dụ image container; mã nguồn là đầu vào. Tự động hóa một quy trình sai có thể khiến lỗi lặp lại nhanh hơn.

Các vai trò và cách chọn hướng học

DevOps Engineer thường chú ý toàn bộ đường đi từ commit đến môi trường chạy: Git, CI/CD, hạ tầng, triển khai và vận hành. Cloud Engineer tập trung thiết kế và quản lý các dịch vụ trên cloud, đặc biệt mạng, IAM, compute, storage và chi phí. SRE tập trung độ tin cậy đo được bằng tín hiệu người dùng, SLO, cảnh báo và xử lý sự cố. Platform Engineer xây nền tảng tự phục vụ để nhóm phát triển dùng các khả năng trên một cách nhất quán. Một công ty nhỏ có thể gộp nhiều vai trò; công ty lớn có thể tách rõ.

Với mục tiêu hiện tại, ta lấy DevOps Engineer làm trục chính. Điều đó có nghĩa bạn cần hiểu ứng dụng đủ để đọc log và test, hiểu Linux và mạng đủ để chẩn đoán, hiểu cloud và hạ tầng đủ để triển khai, rồi

học cách cải thiện độ tin cậy. Sau khi làm một dự án hoàn chỉnh, bạn có thể chọn nhánh Cloud, SRE hoặc Platform để đào sâu.

Đừng chọn công cụ chỉ vì tin tuyển dụng có tên đó. Hãy hỏi: hệ thống đang gặp khó khăn gì, công cụ nào giải quyết được, chi phí và độ phức tạp thêm vào là bao nhiêu? Một ứng dụng nhỏ có thể chạy tốt trên một VM hoặc dịch vụ managed; Kubernetes chưa chắc cần thiết. Khả năng chọn giải pháp vừa đủ là một kỹ năng chuyên môn.

Bài tập

Viết ba câu: (1) kiểu hệ thống bạn muốn xây, (2) điều bạn muốn tự làm được sau ba tháng đầu, (3) loại sự cố bạn muốn biết cách điều tra. Chưa cần đúng thuật ngữ. Câu trả lời này giúp cá nhân hóa bài thực hành.

Đọc sơ đồ một hệ thống trước khi học công cụ

Ta dùng một ứng dụng ghi chú làm câu chuyện xuyên suốt. Người dùng mở trình duyệt, nhập nội dung và bấm Lưu. Trình duyệt gửi một request đến API. API kiểm tra nội dung và yêu cầu database ghi dữ liệu. Database trả kết quả; API trả response; trình duyệt hiển thị ghi chú. Ở thời điểm này chưa cần biết cách lập trình API. Điều cần hiểu là mỗi hộp có một nhiệm vụ và giữa các hộp có kết nối.

Khi hệ thống chạy trên máy của bạn, trình duyệt, API và database có thể cùng ở một máy. Khi chạy trên cloud, chúng có thể nằm ở nhiều máy hoặc dịch vụ khác nhau. Mã ứng dụng không tự tạo được quyền mạng và quyền database. Người học DevOps cần hiểu các điểm nối để biết lỗi xuất hiện ở bước nào. Đây là lý do roadmap bắt đầu bằng hệ điều hành, mạng và Git trước khi học Kubernetes.

Hãy xét ba triệu chứng khác nhau. (1) Trình duyệt không mở được trang: có thể tên miền, mạng hoặc server. (2) Trang mở nhưng bấm Lưu báo lỗi: có thể API, cấu hình database hoặc dữ liệu đầu vào. (3) Bấm Lưu thành công nhưng sau khi restart ghi chú biến mất: có thể dữ liệu chỉ nằm trong bộ nhớ hoặc container tạm. Một người xử lý tốt không đổi cấu hình ngẫu nhiên; họ hỏi câu nào phân biệt được các giả thuyết này.

Bài tập có đáp án hướng dẫn

Giả sử người dùng báo "ứng dụng hỏng". Viết ba câu hỏi bạn sẽ hỏi trước khi sửa. Một câu trả lời hợp lý: bắt đầu hỏng từ khi nào; chức năng nào bị ảnh hưởng và có bao nhiêu người gặp; có thay đổi/deploy nào ngay trước đó không. Ba câu này giúp xác định tác động và mối liên hệ thời gian. Sau đó mới xem log, status và đường request. Câu trả lời khác cũng có thể tốt nếu giúp thu hẹp vấn đề mà không gây thay đổi hệ thống.

Cách tự học một công nghệ mới

Trong DevOps, bạn sẽ liên tục gặp tên công cụ mới. Cách học bền vững là tách **khái niệm**, **cách công cụ thực hiện khái niệm**, và **cú pháp phiên bản hiện tại**. Ví dụ, "dịch vụ cần một địa chỉ ổn định" là vấn đề; Kubernetes Service là một cách giải; tên trường YAML và hành vi phiên bản cụ thể là chi tiết cần tra tài liệu. Nếu học ngược bằng cách chép YAML, bạn có thể chạy được demo nhưng không chẩn đoán được khi nó lỗi.

Mỗi khi gặp thuật ngữ, tạo một thẻ ghi chú năm dòng: nó là gì; giải quyết vấn đề nào; đầu vào/đầu ra; dấu hiệu hoạt động đúng; lỗi thường gặp. Ví dụ với DNS: DNS ánh xạ tên sang địa chỉ; giúp con người dùng tên dễ nhớ; đầu vào là tên, đầu ra thường là bản ghi địa chỉ; kiểm tra bằng công cụ lookup; lỗi thường gặp là record sai hoặc cache chưa cập nhật. Thẻ này chưa đầy đủ nhưng tạo khung để đặt câu hỏi đúng.

Áp dụng phương pháp "dự đoán rồi quan sát" khi làm lab. Trước lệnh **Get-Location**, hãy dự đoán nó trả về gì. Sau khi chạy, so sánh và ghi một câu giải thích. Nếu dự đoán sai, đó là nơi cần học tiếp. Không cần che lỗi khỏi nhật ký; lỗi có giải thích thường hữu ích hơn việc chỉ lưu ảnh màn hình xanh.

Mẫu ghi nhật ký một buổi học

```
Ngay: [ngay hoc]
Muc tieu: Biet terminal dang o thu muc nao
Du doan: Get-Location se in duong dan thu muc hien tai
Lenh da chay: Get-Location
Ket qua thuc te: [chep duong dan cua ban, bo thong tin rieng neu chia se]
Giai thich: Shell chay lenh trong thu muc nay neu dung duong dan tuong doi
Thac mac: Khi doi thu muc, lenh tim file thay doi the nao?
Buoc tiep: Thu Get-ChildItem o hai thu muc khac nhau
```

Ở ví dụ này, các dấu ngoặc vuông là chỗ bạn tự điền. Đừng viết "đã hiểu" nếu chưa giải thích được kết quả. Đây không phải bài chấm điểm; nó là bằng chứng cho lần ôn sau.

Thuật ngữ cần nhớ

- Deployment: đưa một phiên bản ứng dụng vào môi trường chạy.
- CI: tự động kiểm tra và tích hợp thay đổi mã thường xuyên.
- CD: chuẩn bị hoặc tự động đưa phiên bản đã kiểm tra tới môi trường đích, tùy chính sách nhóm.
- Incident: sự kiện làm dịch vụ bị suy giảm hoặc ngừng hoạt động.
- Rollback: quay về trạng thái hoặc phiên bản trước đã biết hoạt động.
- Artifact: sản phẩm build được dùng ở bước sau, như image hoặc gói phần mềm.

Học sâu: một thay đổi đi hết vòng đời như thế nào?

Để hiểu DevOps, hãy theo dõi một thay đổi nhỏ: Notes API cần thêm trường **priority** cho ghi chú. Developer sửa code, thêm migration dữ liệu và test. Git ghi được *ai đổi gì*; CI chứng minh một tập điều kiện tối thiểu; registry chứa image có digest; cấu hình triển khai chọn digest; CD đưa cấu hình lên môi trường; observability đo điều người dùng thật gặp. Nếu chỉ nhớ tên Jenkins, Docker và Kubernetes mà không giải thích được chuỗi này, bạn chưa hiểu DevOps.

Viết một bảng cho từng chặng: **đầu vào**, **đầu ra**, **bằng chứng thành công**, **người chịu trách nhiệm**, **cách lùi khi lỗi**. Ví dụ build có đầu vào là commit và dependency đã khóa, đầu ra là image digest; bằng chứng là build log, test và provenance phù hợp. Deploy có đầu vào là digest được phê duyệt và manifest; đầu ra là workload mới; bằng chứng không chỉ **kubectl apply** exit code 0 mà còn rollout, smoke test và tỷ lệ lỗi sau release. Một chặng không có đầu ra định danh được sẽ khiến việc điều tra sự cố khó hơn.

Năm vòng lặp trong công việc hằng ngày

- **Thay đổi:** nhận yêu cầu, chia thay đổi nhỏ, review và release. Câu hỏi: bản đang chạy đến từ commit nào?
- **Phản hồi:** test, scan và staging cho biết lỗi trước production. Câu hỏi: loại lỗi nào chưa được kiểm tra?
- **Vận hành:** metric/log/trace và alert cho biết dịch vụ thực sự ra sao. Câu hỏi: người dùng bị ảnh hưởng thế nào?
- **Khôi phục:** rollback, failover, restore và runbook giảm thời gian lỗi. Câu hỏi: thao tác nào đã diễn tập?
- **Cải tiến:** postmortem và sửa guardrail ngăn lỗi lặp. Câu hỏi: hệ thống đã thay đổi gì sau sự cố?

Không có đội nào tự động hóa mọi thứ ngay. Chọn công việc lặp lại, dễ sai và có thể kiểm chứng kết quả. Một script restart service nhưng không kiểm tra health chỉ tự động hóa động tác; một workflow xác định mục tiêu, ghi log, kiểm tra sau thay đổi và có đường quay lại mới là tự động hóa vận hành có trách nhiệm.

Bài thực hành trên giấy: thiết kế đường phát hành Notes API

Đầu vào: bạn có Git, Python API, SQLite để học local, Docker và một cluster thử nghiệm. **Bài toán:** mỗi merge vào nhánh chính tạo một image; staging cập nhật tự động; production cần phê duyệt; khi tỷ lệ HTTP 5xx tăng sau release, người trực ca xác định được bản lỗi và khôi phục. Không cần cài phần mềm ở bài này.

Bước 1: vẽ hai repository hoặc hai phần rõ trong một repository: mã nguồn và cấu hình môi trường. Chỉ ra nơi lưu schema migration, Dockerfile, pipeline, manifest và runbook. Nếu dùng một repo, vẫn phải phân quyền đường production.

Bước 2: định nghĩa bốn gate: unit test, integration test với database tạm, render/validate manifest, smoke test sau deploy. Với mỗi gate, ghi lỗi nào nó bắt được và lỗi nào nó không thể bắt. Unit test không phát hiện DNS cloud sai; manifest validation không bảo đảm DB trả lời được.

Bước 3: đặt ID xuyên suốt: commit source, image digest, commit deploy, thời điểm sync và incident ID. Trong một ca lỗi, bạn phải nối được request lỗi với bản phát hành. Nếu chỉ ghi **latest**, bạn không thể tái hiện chính xác artifact.

Bước 4: mô phỏng tình huống: CI xanh, image triển khai, Notes API trả 500 vì migration chưa chạy. Bạn sẽ tạm dừng release ở đâu? Sửa theo kiểu schema tương thích hay rollback? Bằng chứng nào chứng minh phục hồi? Viết câu trả lời trước khi đọc lời giải dưới đây.

Lời giải gợi ý: tạm dừng promotion, xác định bản app và trạng thái schema; giảm tác động bằng rollback chỉ khi schema cũ còn tương thích, nếu không dùng roll-forward/restore theo runbook. Thêm integration test chạy migration trên DB giống cấu hình staging, kiểm tra thứ tự release và smoke test **POST /notes**. Ghi giới hạn: SQLite lab không chứng minh migration MySQL/PostgreSQL production an toàn.

Các thước đo không được hiểu sai

Deployment frequency và lead time giúp thấy tốc độ đưa thay đổi đến người dùng. Change failure rate và thời gian khôi phục giúp thấy độ ổn định của thay đổi. Nhưng nếu ép đội tăng số lần deploy bằng cách chia commit vô nghĩa, bạn đang làm đẹp số chứ không cải thiện hệ thống. Tương tự, “uptime 99,9%” phải nói rõ cửa sổ đo, endpoint, cách tính và người dùng nào. Một tháng 30 ngày có 43.200 phút; 0,1% tương ứng khoảng 43,2 phút không đạt mục tiêu theo cách tính thời gian đơn giản. SLO thực tế có thể dựa trên tỷ lệ request tốt thay vì số phút.

DevOps Engineer, SRE, Cloud Engineer và Platform Engineer giao nhau ở Linux, mạng, tự động hóa và vận hành. DevOps thường tập trung đường delivery và hạ tầng hỗ trợ phát hành; SRE dùng mục tiêu độ tin cậy và kỹ thuật phần mềm cho vận hành; Cloud Engineer thiên về thiết kế/vận hành dịch vụ cloud; Platform Engineer tạo nền tảng self-service cho đội sản phẩm. JD Việt Nam trộn nhiều phạm vi, nên hãy đọc **trách nhiệm thực tế** thay vì chỉ chức danh. JD yêu cầu VMware, SAN, Ceph, Oracle và Azure chuyên sâu cùng một lúc không có nghĩa người mới phải thành thạo tất cả trước khi ứng tuyển.

Câu hỏi có lời giải

Một pipeline xanh có chứng minh release thành công? Không. Nó chỉ chứng minh các điều kiện đã mã hóa trong pipeline. Cần rollout, smoke test theo nghiệp vụ và quan sát sau release.

DevOps có phải một bộ công cụ? Không. Công cụ hiện thực hóa khả năng cộng tác, đo lường, tự động hóa và chịu trách nhiệm đến production. Nếu pipeline tự động nhưng không ai sở hữu phục hồi, vòng đời vẫn đứt.

Khi nào không nên dùng Kubernetes? Khi số workload nhỏ, yêu cầu đơn giản và đội chưa đủ năng lực vận hành, VM hoặc dịch vụ managed có thể ít rủi ro hơn. Quyết định dựa trên chi phí vận hành và yêu cầu thực, không dựa trên độ phổ biến của công cụ.

Bằng chứng học xong: nộp sơ đồ đường phát hành, bảng năm chặng, bốn gate có giới hạn và một phương án khôi phục migration lỗi. Đọc thêm DORA capabilities và Google SRE Workbook.

Một tuần làm việc giả định của DevOps Engineer

Thứ hai, đội sản phẩm muốn release tính năng lọc note. Bạn đọc yêu cầu, xác định app/DB/edge có gì đổi, yêu cầu test và rollback. Thứ ba, bạn giúp developer thêm integration test và image build có digest; không tự viết thay toàn bộ feature. Thứ tư, staging phát hiện query chậm khi dữ liệu lớn; bạn cùng developer xem query plan và thêm index có đo trước/sau. Thứ năm, production deploy theo gate, theo dõi SLI/error rate và giữ khả năng giảm tác động. Thứ sáu, alert báo queue backlog tăng; bạn điều tra consumer, DB, recent deploy, ghi runbook và cải tiến pipeline. Công việc là nối con người, mã nguồn và vận hành bằng bằng chứng, không chỉ “cài tool”.

Toil và tự động hóa đáng làm

Nếu mỗi tuần bạn phải SSH vào 20 host để sửa cùng file Nginx, đó là ứng viên cho Ansible/IaC; nhưng trước khi tự động hóa, cần xác định file owner, cấu hình đúng, thứ tự rollout và smoke test. Nếu mỗi sáng bạn copy metric vào báo cáo, có thể tự động query/visualization. Nếu mỗi incident yêu cầu một quyết định rủi ro mới, không thể biến thành script một dòng; trước tiên viết tiêu chí/gate. Tự động hóa tốt giảm thao tác tay lặp lại và tăng khả năng kiểm tra, chứ không chỉ tăng tốc thay đổi sai.

Bài tập đánh giá một hệ thống “DevOps”

Hệ thống A deploy 20 lần/ngày qua script, nhưng không có test, release ID, rollback hoặc alert. Hệ thống B deploy 2 lần/tuần, mỗi bản có test/approval, digest, smoke, SLO và restore drill. Bạn không thể kết luận A tốt hơn vì nhanh, cũng không thể kết luận B tối ưu vì chậm. Hỏi thêm lead time thực, change failure rate, time to restore, độ phức tạp thay đổi và tác động người dùng. Nếu B có manual gate mất 5 ngày mà không giảm rủi ro, cần cải tiến gate. Nếu A có 20 deploy nhỏ với blast radius thấp và phục hồi nhanh, có thể rất tốt sau khi bổ sung bằng chứng/quan sát. Dữ liệu dẫn quyết định, không phải nhãn “CI/CD hoàn chỉnh”.

Câu hỏi phỏng vấn có lời giải: “Bạn sẽ bắt đầu cải thiện delivery ở công ty mới thế nào?” Trước hết vẽ luồng hiện tại và đo điểm nghẽn/lỗi, chọn một dịch vụ nhỏ có owner, thêm versioned artifact, test có giá trị, deployment record và smoke/rollback, rồi mở rộng. Không khởi đầu bằng việc thay mọi tool hoặc chuyển ngay toàn công ty sang Kubernetes.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://dora.dev/capabilities/
- https://sre.google/workbook/monitoring/